# LAPORAN PRAKTIKUM

# POSTTEST 8

# ALGORITMA PEMROGRAMAN DASAR




**Disusun oleh:**

**Rafa Raisyah Al Fattaah (2609106015)**

**Kelas A1 ‘26**

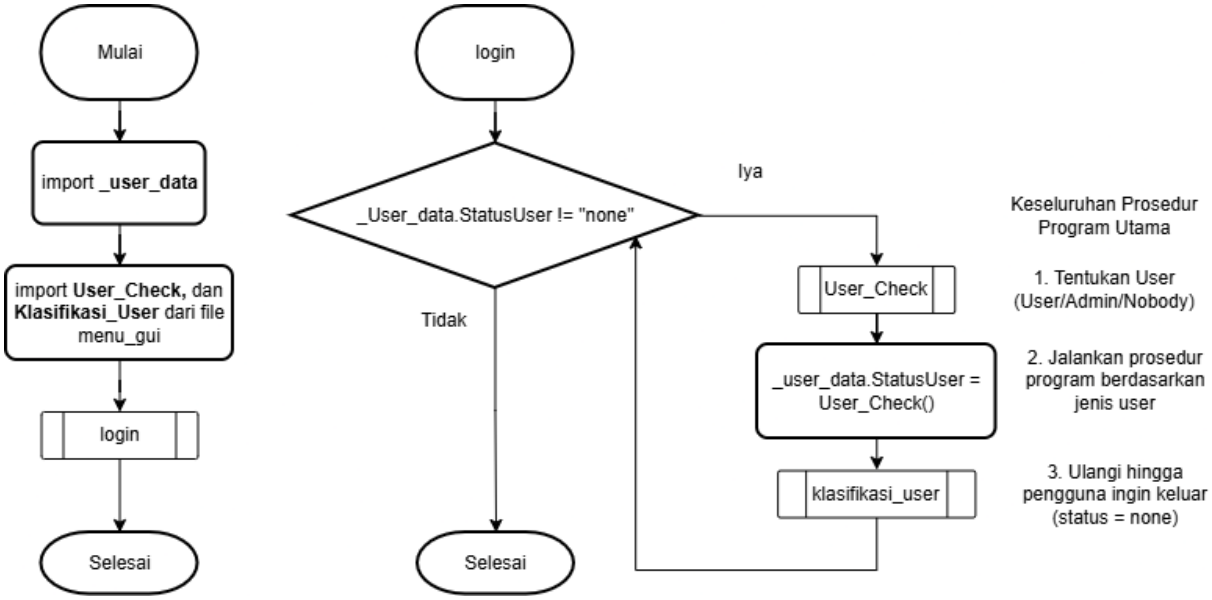
**PROGRAM STUDI INFORMATIKA**

**UNIVERSITAS MULAWARMAN**

**SAMARINDA**

**2026**

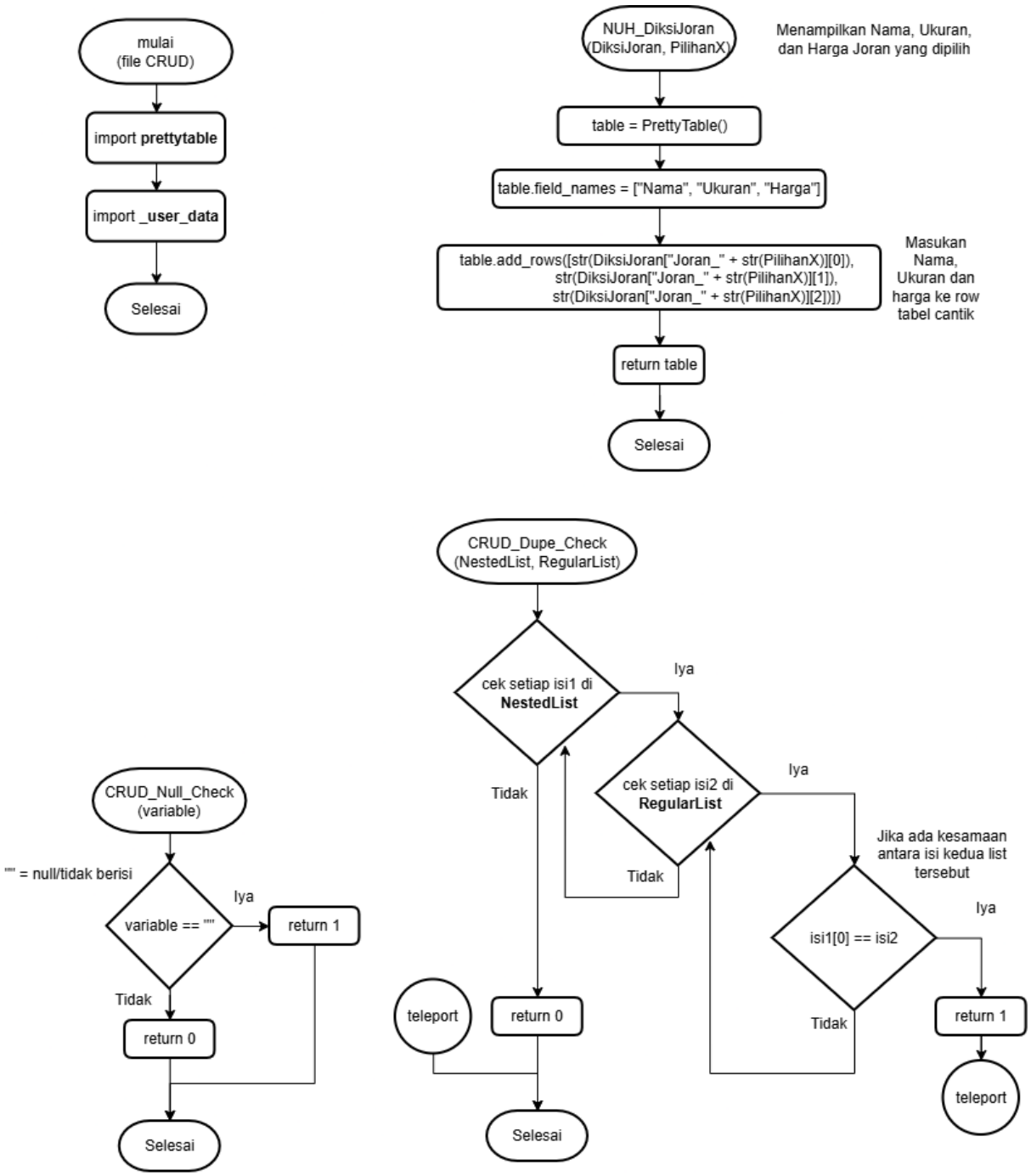
## *1. Flowchart*



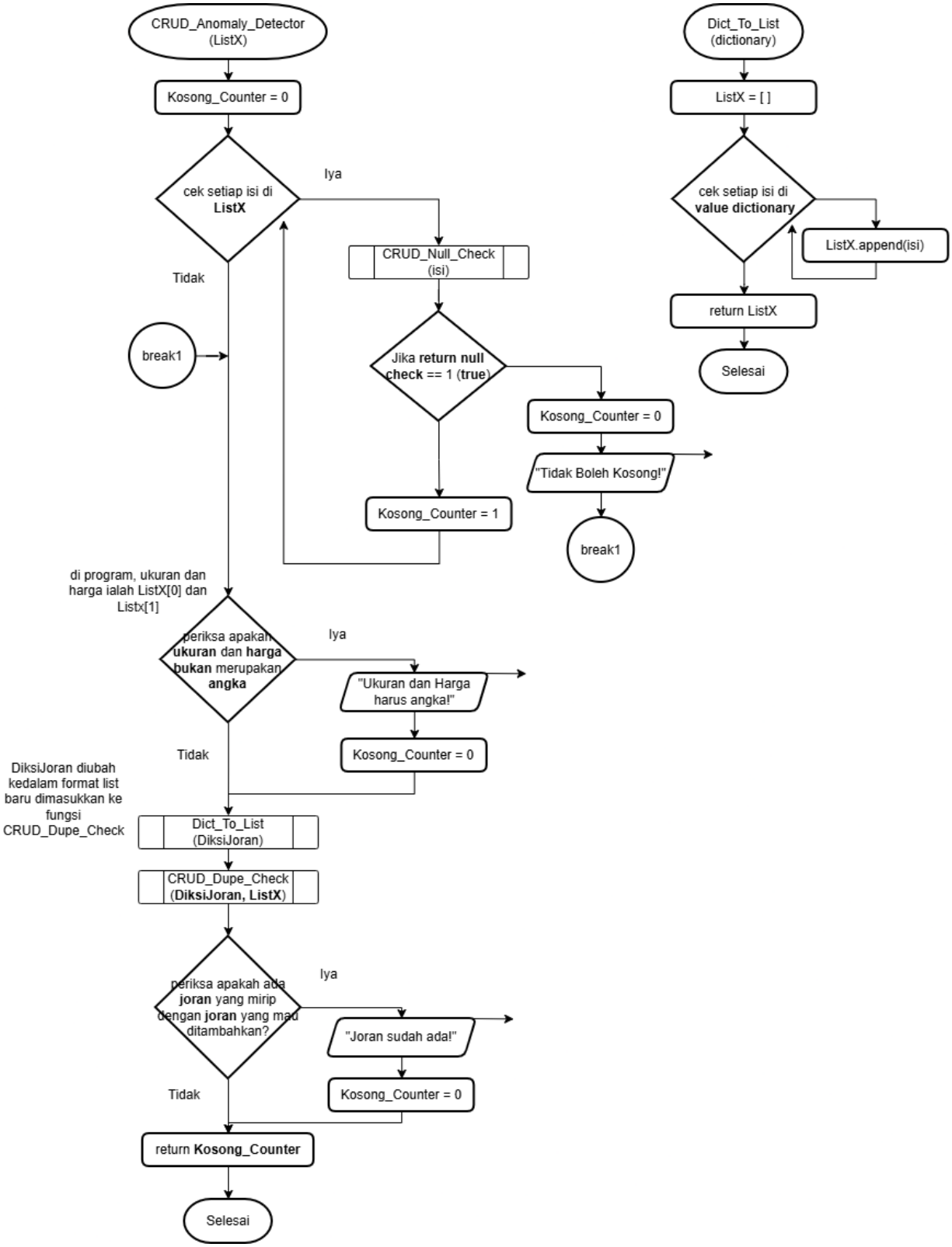

**Gambar 1.1 Program Utama**

Disclaimer:
_user_data digunakan untuk menyimpan data seperti DiksiJoran, UkuranDiksiJoran, StatusUser, DiksiUser, serta username dan password admin

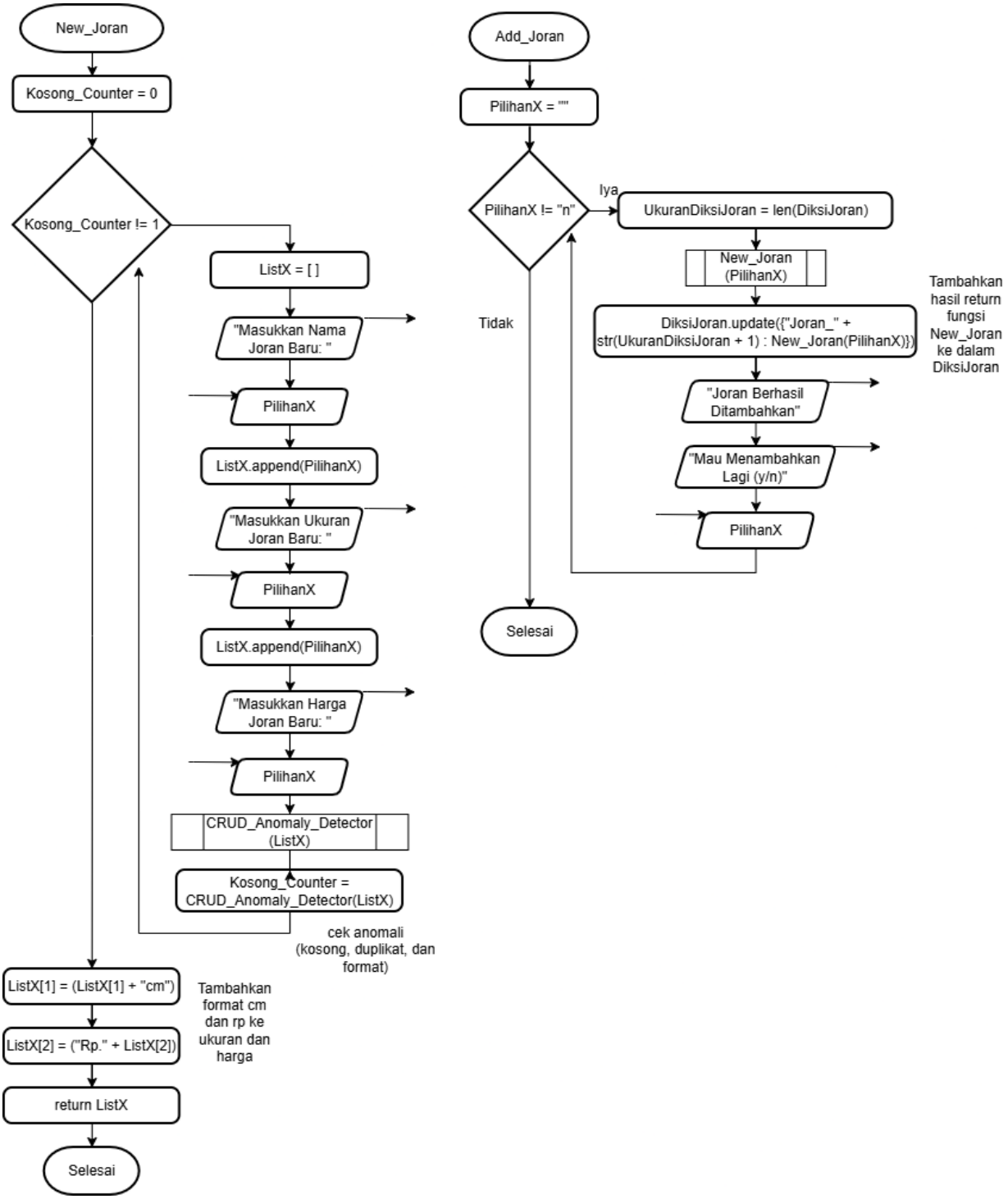
**Gambar 1.2 File _user_data**



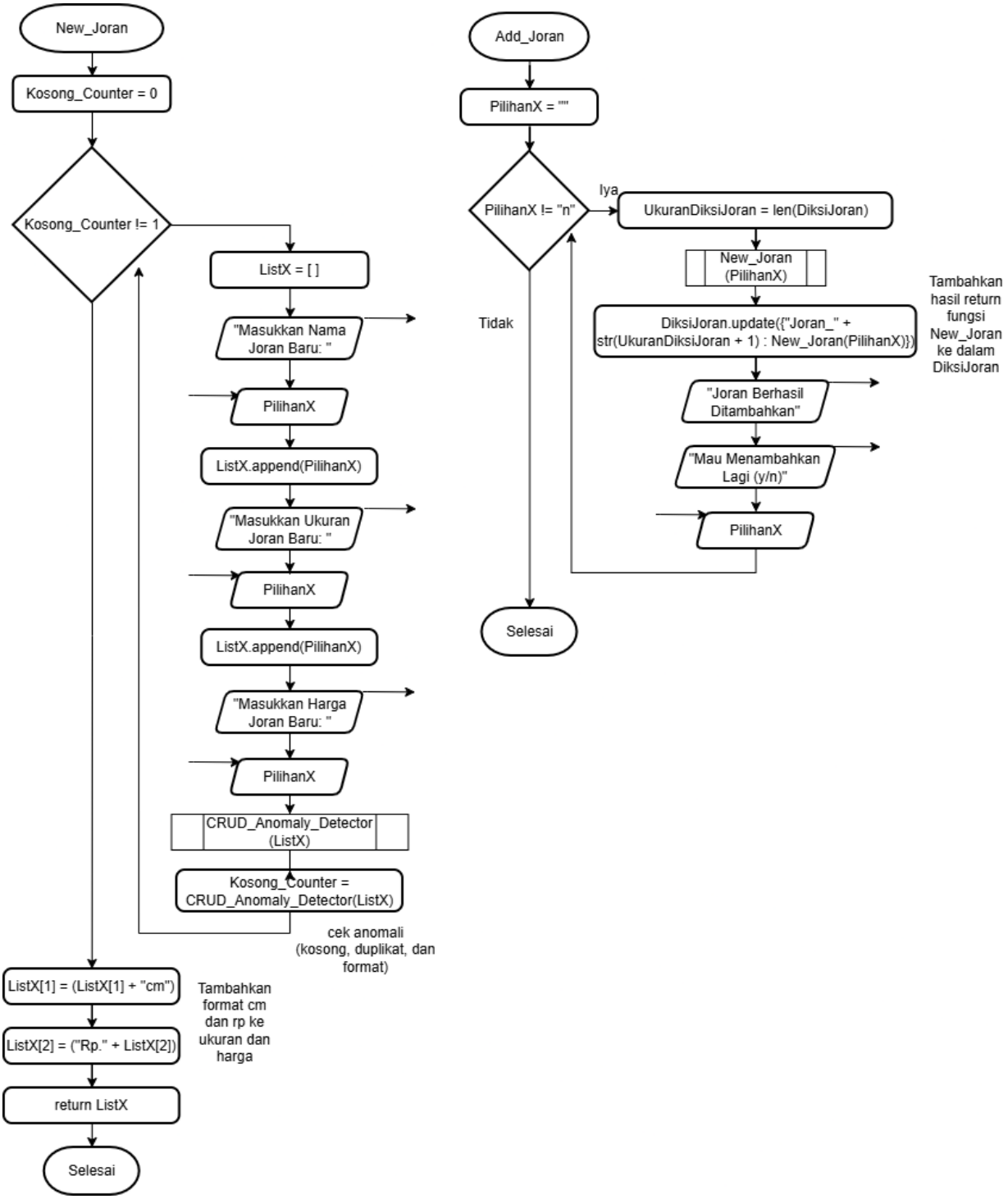

**Gambar 1.3 File CRUD Halaman 1**



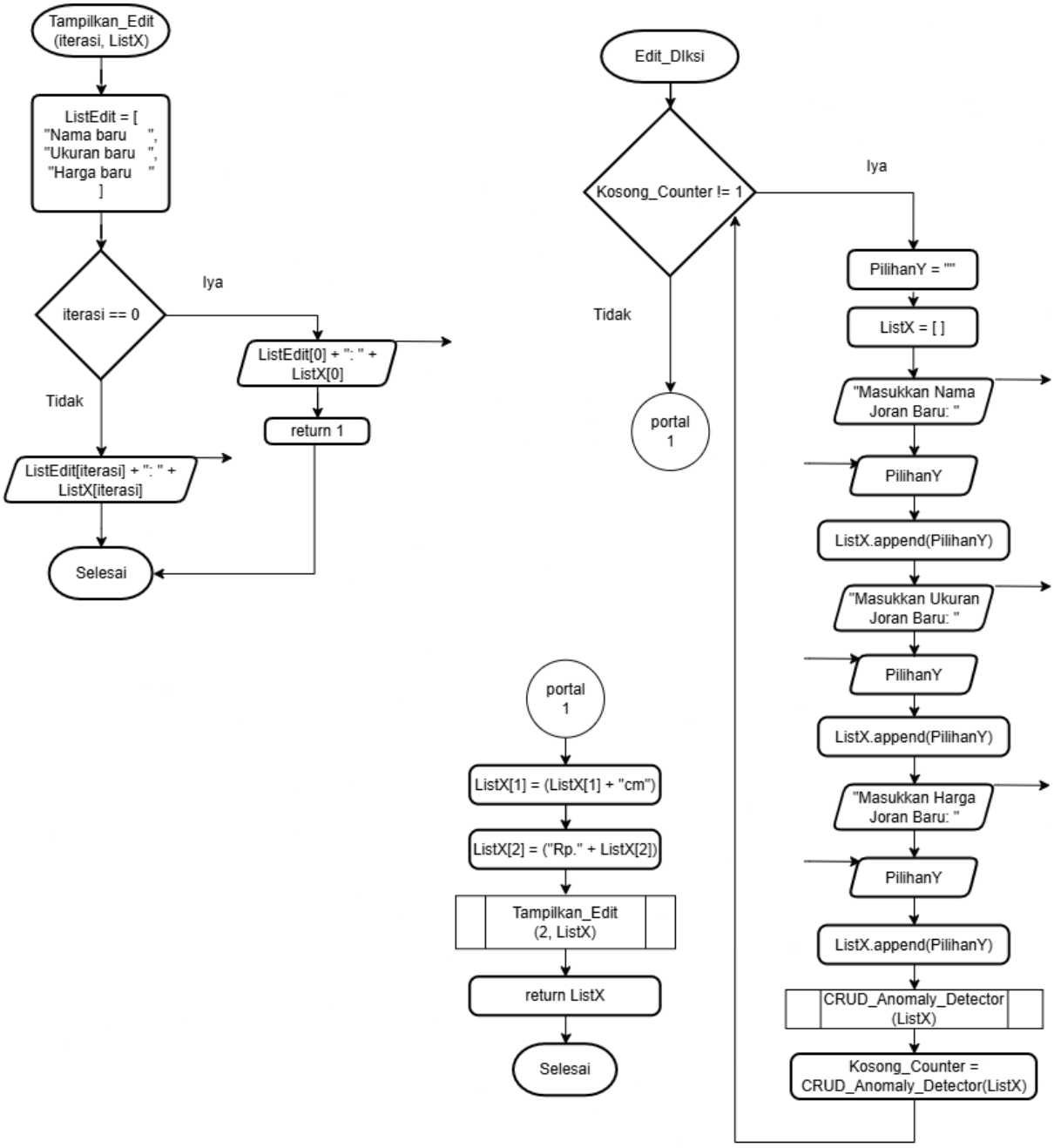

**Gambar 1.4 File CRUD Halaman 2**



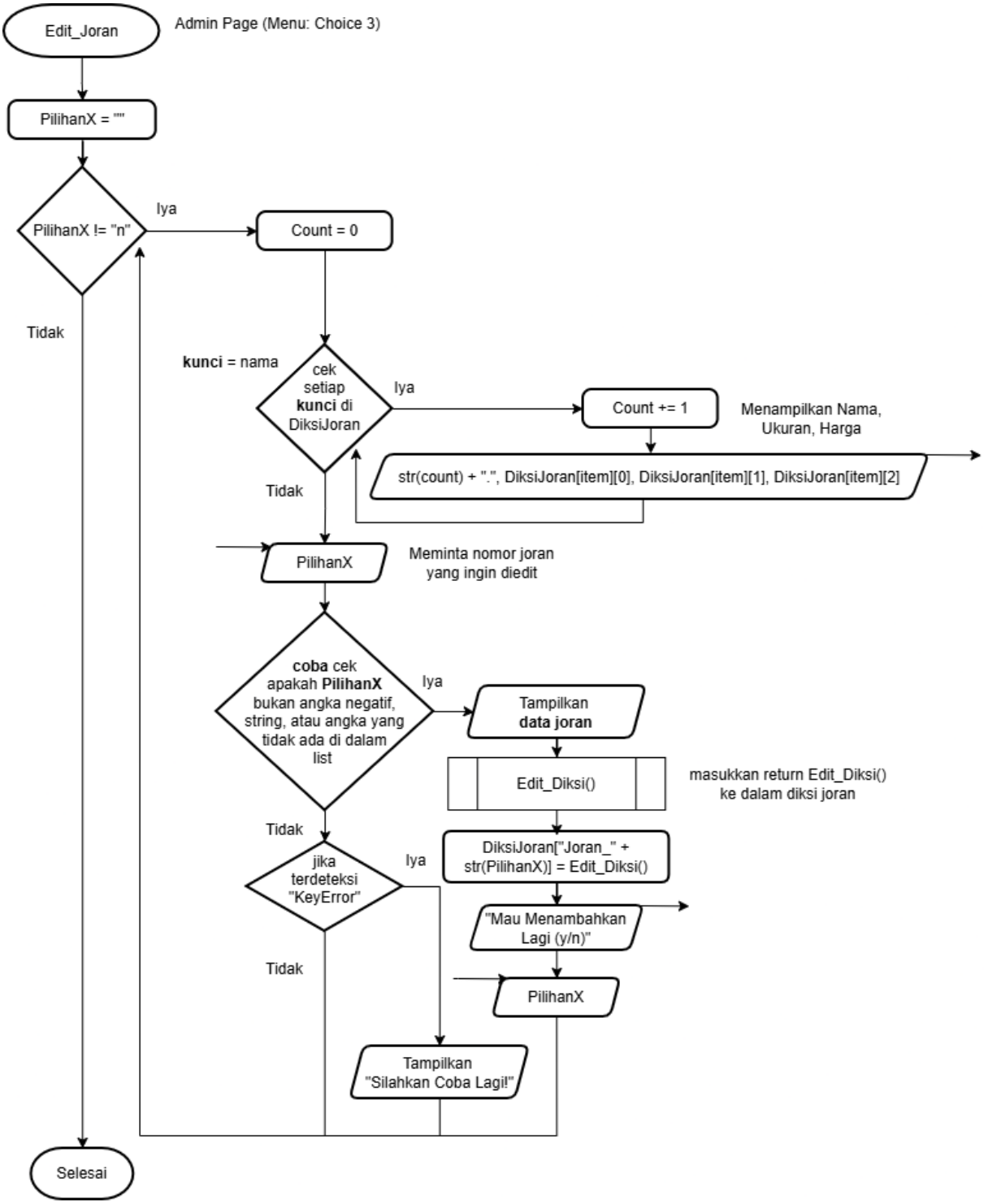

**Gambar 1.5 File CRUD Halaman 3**



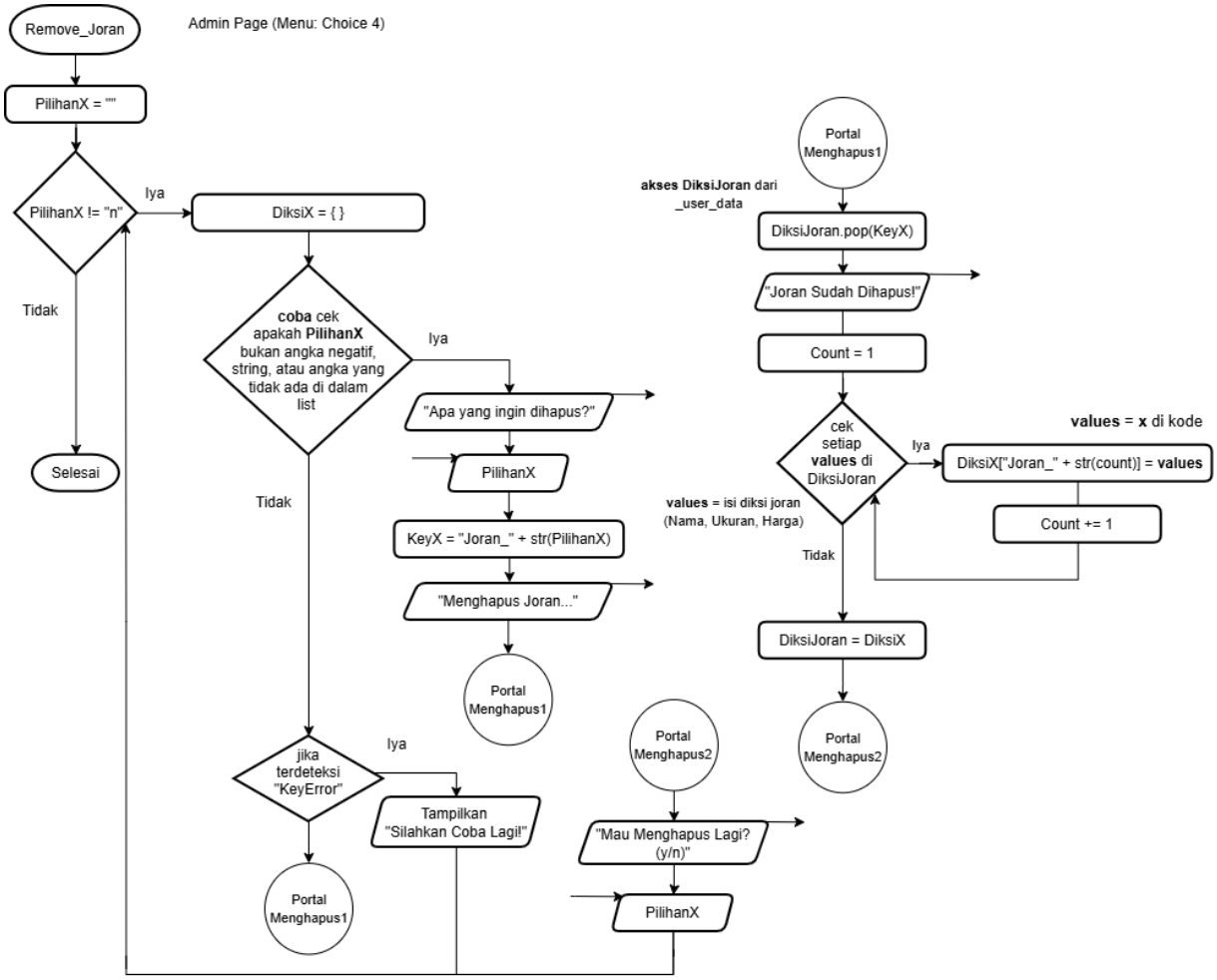

**Gambar 1.6 File CRUD Halaman 4**



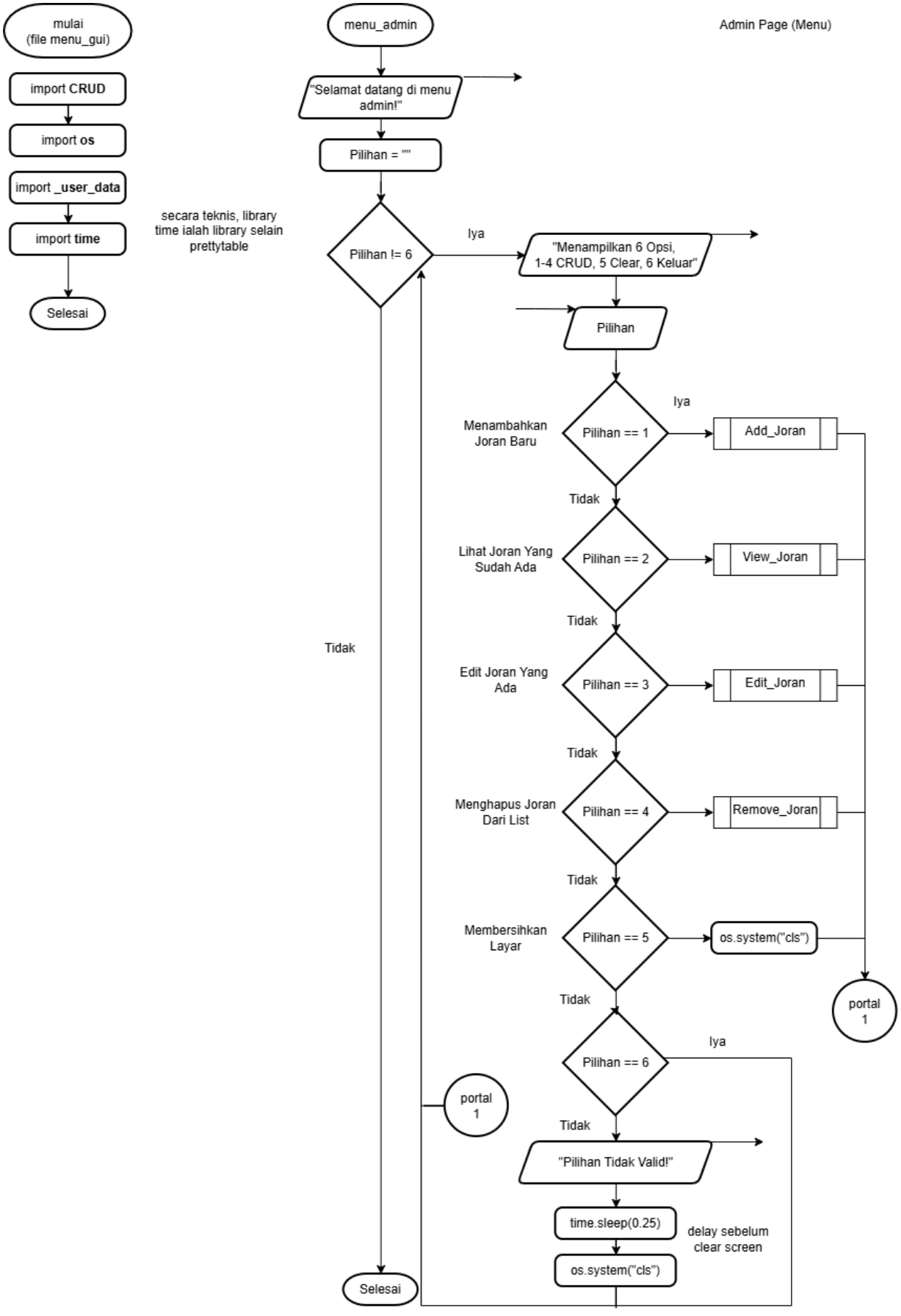

**Gambar 1.7 File CRUD Halaman 5**



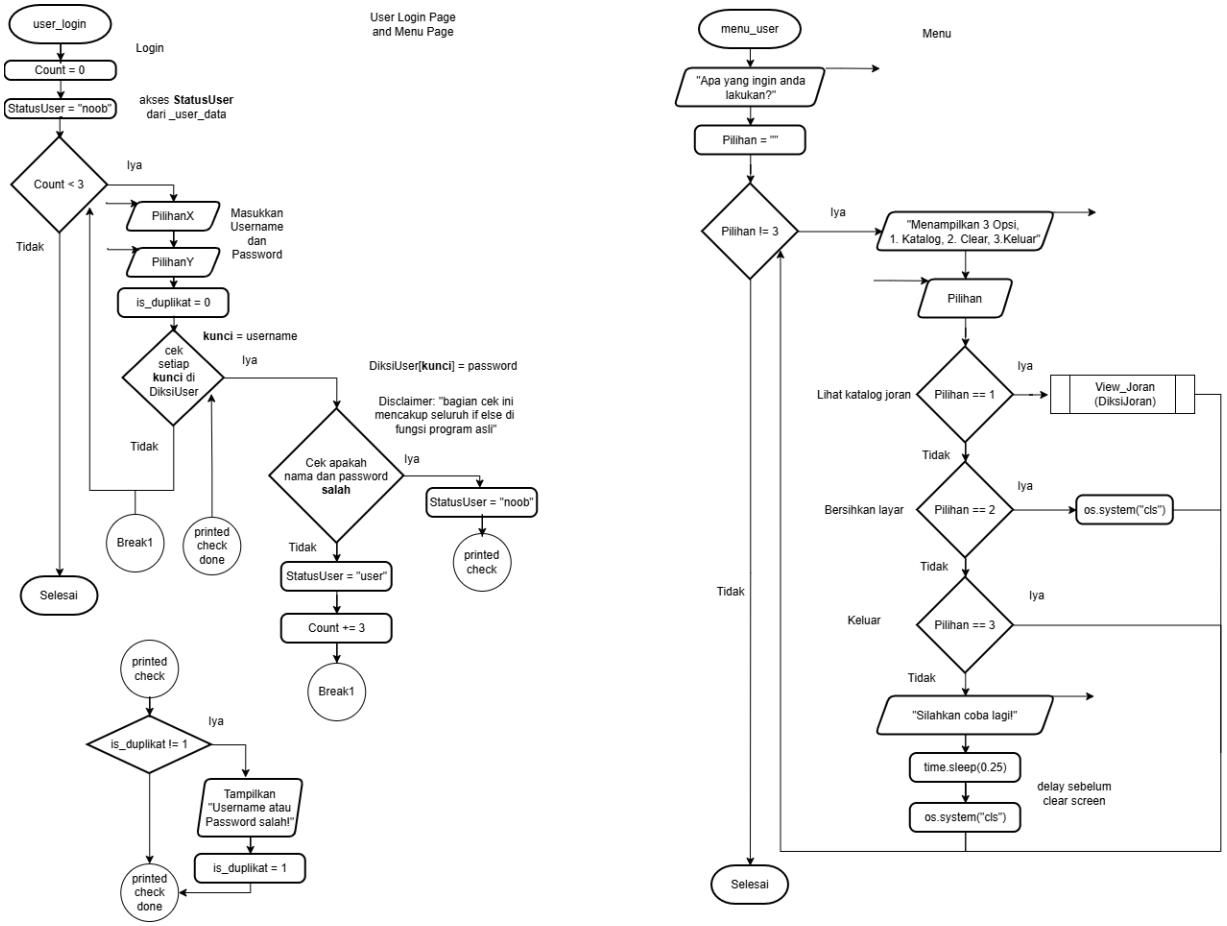

**Gambar 1.8 File CRUD Halaman 6**



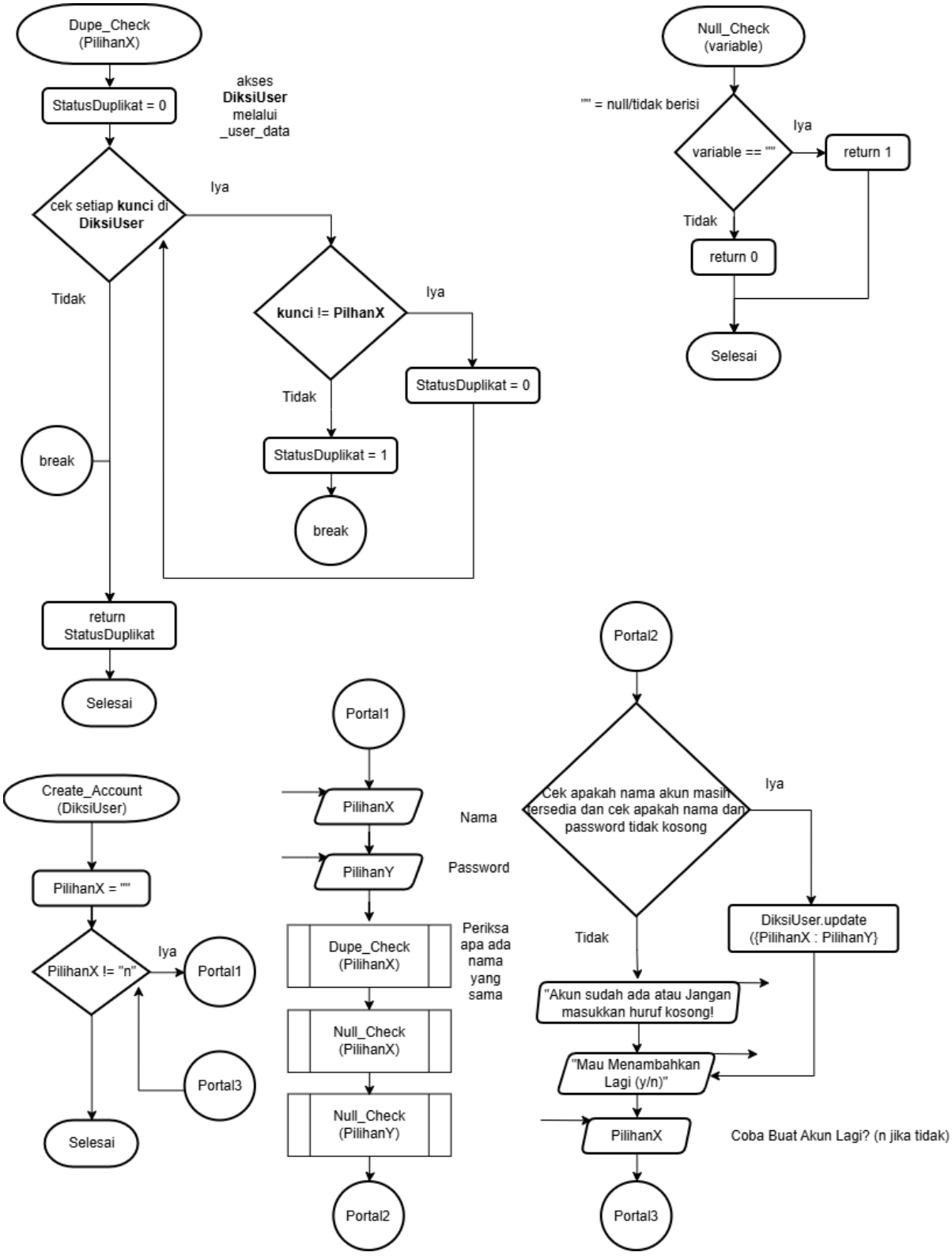

**Gambar 1.9 File CRUD Halaman 7**



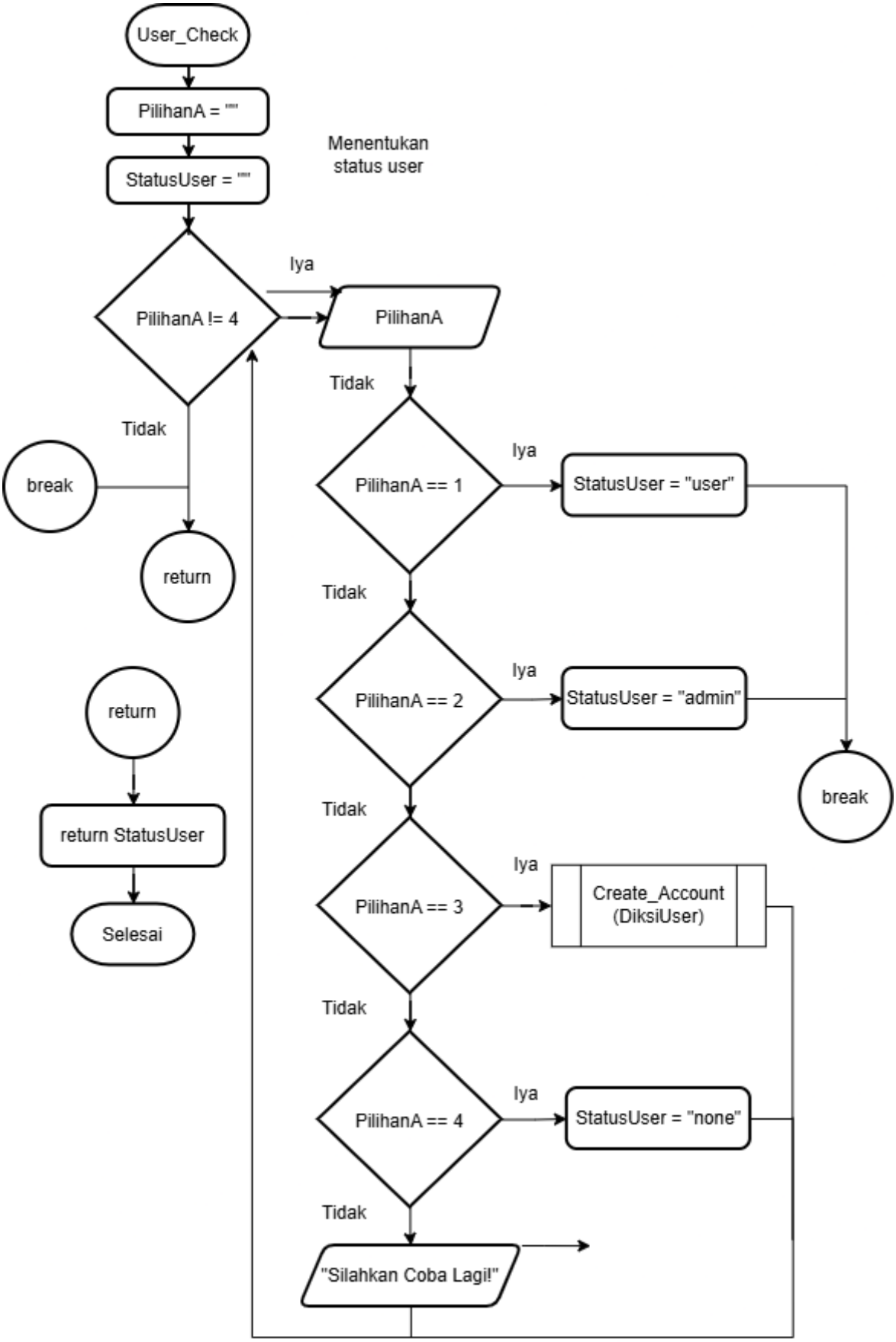

**Gambar 1.10 File menu_gui Halaman 1**



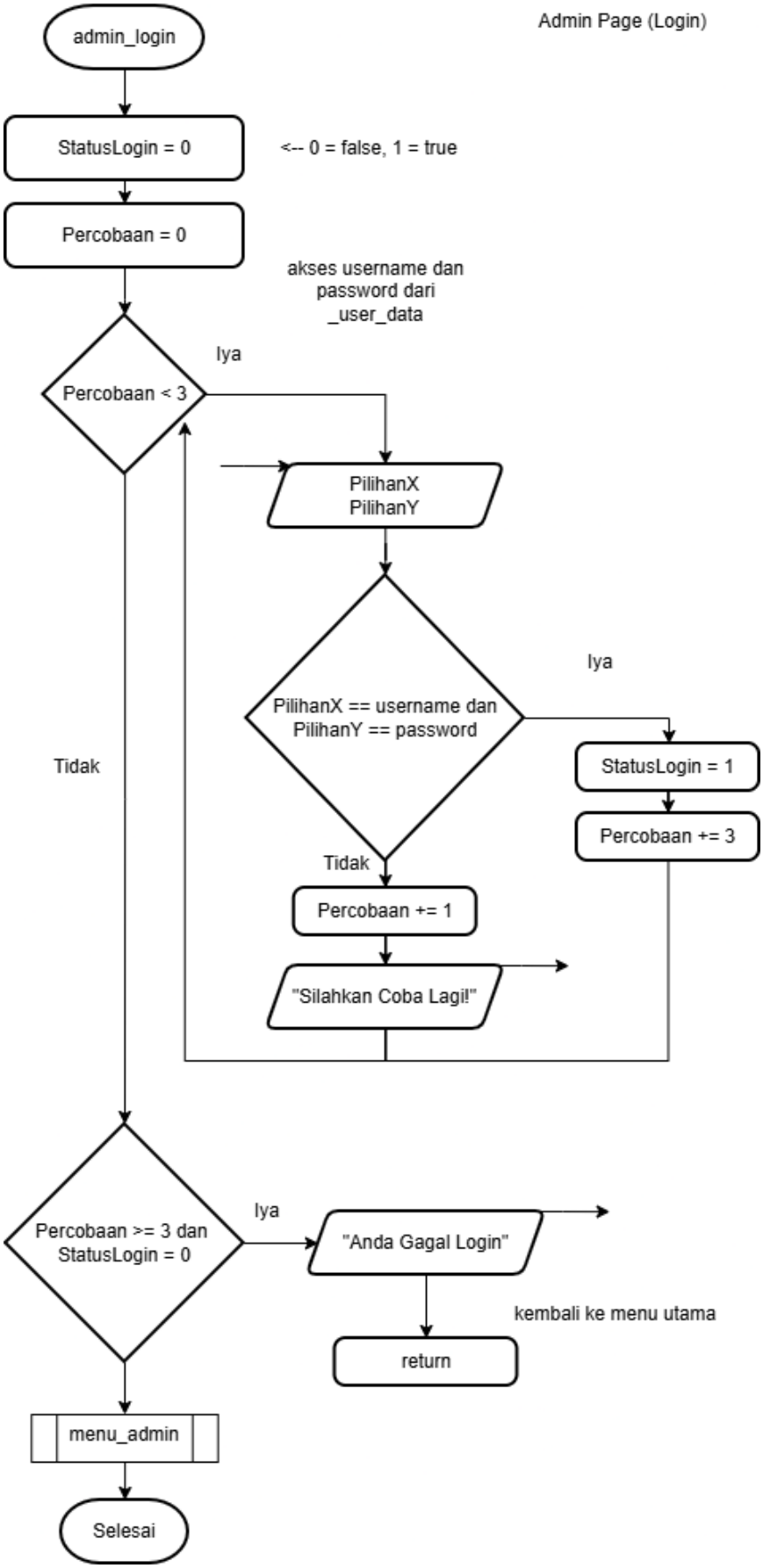

**Gambar 1.11 File menu_gui Halaman 2**



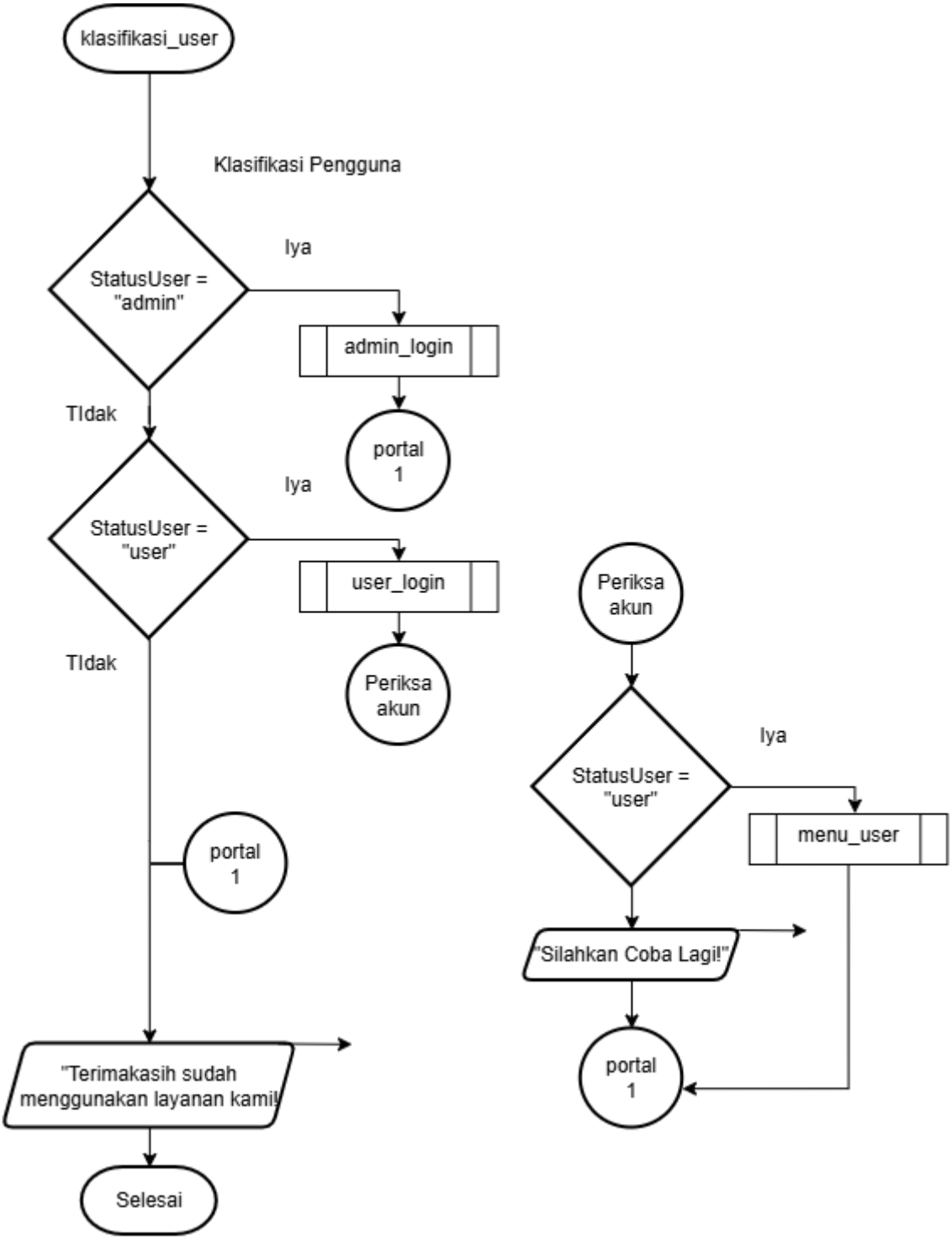

**Gambar 1.12 File menu_gui Halaman 3**




**Gambar 1.13 File menu_gui Halaman 4**




**Gambar 1.14 File menu_gui Halaman 5**




**Gambar 1.15 File menu_gui Halaman 6**

## Penjelasan Flowchart

Flowchart atau bagan alur adalah diagram yang dirancang untuk memvisualisasikan langkah-langkah, proses, keputusan, dan alur kerja suatu sistem secara terstruktur (sistematis), logis, dan jelas. Setiap langkah dalam flowchart digambarkan dalam bentuk diagram dan dihubungkan dengan garis atau arah panah. Fungsi utama dari flowchart adalah memberikan gambaran jalannya sebuah program dari satu proses ke proses lainnya hingga selesai. Selain itu, flowchart juga digunakan untuk menyederhanakan rangkaian prosedur agar lebih mudah dipahami.

Flowchart tersebut menggambarkan alur kerja program katalog joran pancing digital yang menerapkan prinsip CRUD (Create, Read, Update, Delete). Program ini dibuat untuk membantu pengelolaan data joran pancing yang terdiri dari nama, ukuran, dan harga. Dalam penggunaannya, program memanfaatkan percabangan dan perulangan untuk mengatur pilihan pengguna, memvalidasi data yang dimasukkan, serta membedakan hak akses antara admin dan guest. Admin memiliki hak untuk menambahkan, melihat, mengedit, dan menghapus data joran, sedangkan guest hanya dapat melihat katalog yang tersedia.

Pada tahap awal, program mempersiapkan beberapa variabel yang dibutuhkan untuk menjalankan setiap prosesnya. Data utama joran disimpan dalam variabel DiksiJoran yang menggunakan struktur dictionary. Setiap data memiliki key dengan format "Joran_" yang diikuti nomor urut, sedangkan isi dari setiap key berupa list yang menyimpan nama, ukuran, dan harga joran. Program juga menggunakan fungsi len() untuk mengetahui jumlah data yang tersedia. Selain itu, variabel StatusUser digunakan untuk menentukan jenis pengguna yang sedang mengakses program, sementara informasi username dan password digunakan dalam proses autentikasi admin.

Selanjutnya, program menampilkan menu awal dan meminta pengguna memasukkan pilihan melalui variabel PilihanA. Input tersebut kemudian diperiksa menggunakan percabangan untuk menentukan halaman yang akan diakses. Jika pengguna memilih angka 1, program akan mengarahkan pengguna ke menu guest. Jika memilih angka 2, pengguna akan diarahkan ke proses login admin. Pilihan lainnya digunakan untuk proses pembuatan akun atau keluar dari program, sesuai dengan menu yang tersedia. Apabila input tidak sesuai dengan pilihan yang diberikan, program akan menampilkan pesan kesalahan dan meminta pengguna memasukkan pilihan kembali.

Apabila pengguna memilih masuk sebagai admin, program akan menjalankan proses login dengan meminta username dan password. Input tersebut kemudian dibandingkan dengan informasi akun yang telah ditentukan. Program menggunakan variabel StatusLogin untuk menandai keberhasilan proses login dan Percobaan untuk menghitung jumlah percobaan yang telah dilakukan. Jika username dan password sesuai, pengguna dapat melanjutkan ke menu admin. Namun, jika informasi yang dimasukkan salah, program akan memberikan kesempatan untuk mencoba kembali hingga batas percobaan yang ditentukan, yaitu tiga kali. Apabila seluruh percobaan gagal, pengguna tidak dapat melanjutkan ke menu admin. Proses ini bertujuan untuk memastikan bahwa hanya pengguna yang memiliki akses admin yang dapat mengelola data katalog.

Setelah berhasil login, program akan menampilkan menu admin yang menyediakan enam pilihan. Variabel Pilihan digunakan untuk menampung input pengguna, sedangkan perulangan akan terus berjalan selama admin belum memilih opsi untuk keluar. Pilihan pertama digunakan untuk menambahkan joran baru, pilihan kedua untuk melihat katalog, pilihan ketiga untuk mengedit data, pilihan keempat untuk menghapus data, pilihan kelima untuk membersihkan layar, dan pilihan keenam untuk keluar dari menu admin. Jika pengguna memasukkan pilihan yang tidak tersedia, program akan menampilkan pesan kesalahan dan meminta pengguna memilih kembali.

Jika admin memilih pilihan pertama, program akan menjalankan proses penambahan data joran. Proses ini ditangani oleh fungsi Add_Joran() yang menggunakan fungsi New_Joran() untuk menerima data baru. Program terlebih dahulu membuat ListX sebagai list kosong untuk menampung nama, ukuran, dan harga joran. Setiap input yang dimasukkan admin disimpan menggunakan perintah ListX.append(). Setelah seluruh data terkumpul, program menjalankan fungsi CRUD_Anomaly_Detector() untuk memeriksa apakah terdapat input kosong, ukuran atau harga yang bukan angka, serta nama joran yang sudah terdaftar. Jika seluruh pemeriksaan berhasil dilewati, ukuran dan harga akan ditambahkan format satuannya, yaitu "cm" dan "Rp.". Data tersebut kemudian dimasukkan ke dalam dictionary DiksiJoran menggunakan fungsi .update() dengan key baru seperti "Joran_1", "Joran_2", dan seterusnya. Admin selanjutnya dapat memilih untuk menambahkan joran lainnya atau kembali ke menu utama. Proses ini merupakan penerapan prinsip Create karena program menambahkan data baru ke dalam katalog.

Jika admin memilih pilihan kedua, program akan menampilkan seluruh data joran yang telah tersimpan. Proses ini dijalankan melalui fungsi View_Joran(). Program melakukan perulangan untuk membaca setiap data di dalam dictionary DiksiJoran, kemudian menampilkan nama joran beserta nomor urutnya. Admin dapat memilih salah satu joran untuk melihat informasi lebih lengkap. Untuk menampilkan detail tersebut, program menggunakan fungsi NUH_DiksiJoran() yang memanfaatkan library PrettyTable. Fungsi ini membuat tabel dengan tiga kolom, yaitu nama, ukuran, dan harga, kemudian mengambil informasi berdasarkan key joran yang dipilih. Jika nomor yang dimasukkan tidak sesuai dengan key yang tersedia, program akan menampilkan pesan kesalahan. Proses ini merupakan penerapan prinsip Read karena program hanya membaca dan menampilkan data tanpa mengubah informasi yang tersimpan.

Jika admin memilih pilihan ketiga, program akan menjalankan proses pengeditan data joran melalui fungsi Edit_Joran(). Program terlebih dahulu menampilkan seluruh data joran beserta nomor urutnya agar admin dapat menentukan data yang ingin diubah. Setelah admin memasukkan nomor joran, program akan mencari data berdasarkan key yang sesuai, misalnya "Joran_1". Jika data ditemukan, informasi joran tersebut akan ditampilkan sebelum proses pengeditan dimulai. Selanjutnya, fungsi Edit_Diksi() meminta admin memasukkan nama, ukuran, dan harga yang baru. Data tersebut ditampung sementara ke dalam ListX, kemudian diperiksa menggunakan fungsi CRUD_Anomaly_Detector() untuk memastikan tidak ada input kosong, kesalahan format angka, maupun nama yang terduplikasi. Setelah data dinyatakan valid, program menyesuaikan format ukuran dan harga, kemudian menampilkan hasil perubahan menggunakan fungsi Tampilkan_Edit(). Data baru selanjutnya digunakan untuk menggantikan data lama pada key dictionary yang dipilih. Proses ini merupakan penerapan prinsip Update karena program memperbarui informasi joran yang sebelumnya sudah tersimpan.

Jika admin memilih pilihan keempat, program akan menjalankan proses penghapusan data joran melalui fungsi Remove_Joran(). Program meminta admin memasukkan nomor joran yang ingin dihapus melalui variabel PilihanX. Input tersebut kemudian digabungkan dengan teks "Joran_" untuk membentuk key yang akan dicari di dalam dictionary DiksiJoran. Jika key tidak ditemukan, program akan menampilkan pesan bahwa data joran tidak tersedia. Sebaliknya, jika data ditemukan, program menampilkan informasi joran yang akan dihapus, kemudian menghapusnya menggunakan perintah .pop(). Setelah penghapusan berhasil,

program menyusun kembali urutan key dictionary mulai dari "Joran_1" menggunakan perulangan dan dictionary sementara bernama DiksiX. Dengan demikian, nomor urut joran tetap tersusun secara berurutan meskipun salah satu data telah dihapus. Admin kemudian dapat memilih untuk menghapus data lainnya atau kembali ke menu utama. Proses ini merupakan penerapan prinsip Delete karena program menghilangkan data yang dipilih dari katalog.

Pilihan kelima pada menu admin digunakan untuk membersihkan tampilan terminal. Ketika pilihan ini dipilih, program menjalankan perintah os.system("cls") untuk menghapus tampilan sebelumnya sehingga terminal kembali terlihat bersih. Setelah itu, program akan kembali menampilkan menu admin. Sementara itu, pilihan keenam digunakan untuk keluar dari menu admin. Ketika admin memilih angka 6, kondisi perulangan menu tidak lagi terpenuhi sehingga program menghentikan proses tersebut dan melanjutkan ke alur berikutnya.

Di sisi lain, apabila pengguna memilih menjadi guest pada menu awal, program akan langsung mengarahkan pengguna ke menu pengunjung tanpa melalui proses login admin. Menu guest menyediakan tiga pilihan, yaitu melihat katalog joran, membersihkan layar, dan keluar dari program. Jika guest memilih pilihan pertama, program akan menampilkan katalog menggunakan proses pembacaan data yang serupa dengan menu admin. Program membaca setiap data di dalam dictionary DiksiJoran, kemudian menampilkan nama, ukuran, dan harga joran yang tersedia. Guest juga dapat memilih joran tertentu untuk melihat informasi lebih lengkap melalui fungsi NUH_DiksiJoran() yang menampilkan data dalam bentuk tabel menggunakan PrettyTable. Jika nomor yang dimasukkan tidak sesuai dengan data yang tersedia, program akan menampilkan pesan kesalahan. Dengan demikian, guest tetap dapat memperoleh informasi katalog tanpa memiliki akses untuk mengubah data di dalamnya.

Jika guest memilih pilihan kedua, program akan menjalankan perintah os.system("cls") untuk membersihkan tampilan terminal, kemudian kembali ke menu pengunjung. Namun, jika guest memilih pilihan ketiga, program akan keluar dari menu tersebut dan melanjutkan ke proses akhir. Berbeda dengan admin, guest tidak diberikan akses untuk menambahkan, mengedit, maupun menghapus data joran. Pembatasan ini diterapkan agar pengelolaan katalog tetap berada di bawah kendali admin.

Secara keseluruhan, program menggunakan beberapa variabel dan fungsi yang saling berhubungan untuk mendukung proses pengelolaan katalog. Variabel PilihanA digunakan untuk menentukan pilihan pada menu awal, sedangkan StatusUser berfungsi untuk

membedakan jenis pengguna yang sedang mengakses program. Variabel Pilihan digunakan untuk menerima pilihan menu, sementara PilihanX dan PilihanY berfungsi sebagai penampung input sementara. Dictionary DiksiJoran menjadi tempat penyimpanan utama seluruh data joran, sedangkan ListX digunakan untuk menampung data baru atau data hasil pengeditan sebelum dimasukkan ke dalam dictionary. Selain itu, fungsi dict_to_list() digunakan untuk mengubah isi dictionary menjadi list agar lebih mudah diperiksa oleh fungsi validasi. Program juga menggunakan CRUD_Null_Check(), CRUD_Dupe_Check(), dan CRUD_Anomaly_Detector() untuk memastikan data yang dimasukkan sesuai dengan ketentuan. Sementara itu, PrettyTable digunakan untuk menyajikan informasi joran dalam bentuk tabel agar lebih mudah dibaca.

Pada tahap akhir, ketika pengguna memilih untuk keluar dari program, alur akan menuju proses penutupan. Program kemudian menampilkan pesan ucapan terima kasih sebagai penanda bahwa penggunaan katalog telah selesai. Dengan demikian, program katalog joran pancing digital ini tidak hanya menyediakan fitur untuk menyimpan dan menampilkan informasi joran, tetapi juga mempermudah admin dalam mengelola data melalui penerapan prinsip CRUD. Sementara itu, guest tetap dapat melihat informasi yang tersedia tanpa mengubah isi katalog.

## 2. Deskripsi Singkat Program

Secara singkat, program ini dirancang untuk mempermudah interaksi antara Pemilik toko pancingan dengan kostumer online. Pembagian sistem admin dan user(user) dibuat supaya program bisa digunakan oleh kedua belah pihak.

Program memanfaatkan data dari barang jualan toko yang sudah disimpan, dan Memberikan kebebasan bagi admin untuk membuat, membaca, mengubah, maupun menghapus barang tersebut dari katalog toko digital. Untuk pengguna, mereka hanya diberikan fitur untuk melihat katalog toko, mereka tidak bisa melakukan aksi yang mengubah struktur toko.

3. **Pseudo Code**

# File main

```
import _user_data
import menu_gui
#Variabel

#Program Utama
def login():
    while _user_data.StatusUser != "none":
        _user_data.StatusUser = menu_gui.User_Check()
        menu_gui.klasifikasi_user()

login()
```

# File _user_data

```
#User Database
DiksiJoran = { #Diksi Joran
            'Joran_1'   : ['shimano', '150cm', 'RP.200000'], #<---------
Termasuk Nested List
            'Joran_2'   : ['daiwa', '130cm', 'Rp.125000'],
            'Joran_3'   : ['relix nusantara','150cm', 'RP.200000'],
            'Joran_4'   : ['daido' , '150cm', 'RP.200000'],
            'Joran_5'   : ['sougayilang', '150cm', 'RP.200000']
    }

UkuranDiksiJoran = len(DiksiJoran)
StatusUser = ""

#Username, Password
DiksiUser = {"Daniel" : "Danie123"}


#Data Admin
username = "John Batubara"
password = "Tongkang123"
```

# File CRUD

```
import _user_data
from prettytable import PrettyTable
#Tidak bisa import menu gui jadi fungsi null sy pindahkan

def NUH_DiksiJoran(DiksiJoran : int, PilihanX : int):
    table = PrettyTable()
    table.field_names = ["Nama", "Ukuran", "Harga"]
    table.add_row([DiksiJoran["Joran_" + str(PilihanX)][0],
                   DiksiJoran["Joran_" + str(PilihanX)][1],
                   DiksiJoran["Joran_" + str(PilihanX)][2]])
    return table

def CRUD_Null_Check(variables : str):
    if variables == "":
        return 1
    return 0

def CRUD_Dupe_Check(List1, List2):
    for isi1 in List1:
        for isi2 in List2:
            if isi1[0] == isi2:
                return 1
    return 0

def CRUD_Anomaly_Detector(ListX):
    #Cek Kosongan
    Kosong_Counter = 0
    for isi in ListX:
        if CRUD_Null_Check(isi) == 1:
            Kosong_Counter = 0
            print("Jangan masukkan huruf kosong!, silahkan coba lagi!")
            break
        else:
            Kosong_Counter = 1
    #Cek Digitan
    if not ListX[1].isdigit() or not ListX[2].isdigit():
        print("Dibagian Ukuran dan Harga tolong masukkan angka yang sesuai!")
        Kosong_Counter = 0

    #Cek Duplikat
    if CRUD_Dupe_Check(dict_to_list(_user_data.DiksiJoran), ListX):
        print ("Terdeteksi adanya kesamaan dengan joran yang sudah ada!\n" \
               "Silahkan Coba Lagi!")
        Kosong_Counter = 0
    return Kosong_Counter
```

```
def dict_to_list(dictionary : dict):
    ListX = []
    for isi in dictionary.values():
        ListX.append(isi)
    return ListX

#Pilihan 1
def New_Joran(PilihanX):
    Kosong_Counter = 0

    while Kosong_Counter != 1:
        ListX = []
        PilihanX = input("Masukkan Nama Joran Baru: ")
        ListX.append(PilihanX)
        PilihanX = input("Masukkan Ukuran Joran (cm): ")
        ListX.append(PilihanX)
        PilihanX = input("Masukkkan Harga Joran (Rp): ")
        ListX.append(PilihanX)
        #Cek Kosongan
        Kosong_Counter = CRUD_Anomaly_Detector(ListX)

    #Perbaiki Format
    ListX[1] = (ListX[1] + "cm")
    ListX[2] = ("Rp." + ListX[2])
    return ListX

def Add_Joran(DiksiJoran : dict):
    PilihanX = ""
    while PilihanX != 'n':
        UkuranDiksiJoran = len(DiksiJoran)
        DiksiJoran.update({"Joran_" + str(UkuranDiksiJoran + 1) :
New_Joran(PilihanX)})
        print("Joran Berhasil ditambahkan ke dalam katalog!")
        PilihanX = input("apakah anda ingin menambahkan joran lagi?\n" \
                         "ketik 'y' atau huruf lain jika iya\n" #disini
kenapa eror pas '\' diletakan, saya tidak tau
                         "ketik 'n' jika tidak: " )
                         #Jika itu bekerja, jangan disentuh
#Pilihan 1

#Pilihan 2

def View_Joran(DiksiJoran : dict):
    PilihanX = ""
    while PilihanX != "n":
        print ("Nama-Nama Joran di Inventori")
        count = 0
        for key in DiksiJoran:
```

```
            count += 1
            print(str(count) + ".", DiksiJoran[key][0])

        PilihanX = input("Untuk mengetahui lebih lanjut mengenai joran, ketik
'y', " \
                         "Untuk kembali ke menu, ketik 'n'\n" \
                         "Pilihan                          : ")
        if PilihanX == 'y':
            while PilihanX != "n":
                PilihanX = input(
                             "Pilih angka Joran               : ")
                try:
                    print (NUH_DiksiJoran(DiksiJoran, PilihanX))
                    PilihanX = input("Ketik huruf apapun untuk lanjut, huruf
'n' untuk ke menu utama: ")
                    if PilihanX == 'n':
                        break
                except KeyError:
                    print ("Jangan masukkan angka negatif, string, atau angka
yang tidak ada didalam dictionary!")
        elif PilihanX != "n":
            print ("Jangan masukkan huruf lain selain (y/n)")
#Pilihan 2

#Pilihan 3

def Tampilkan_Edit(iterasi: int, ListX : list):
    ListEdit = ["Nama baru     ", "Ukuran baru   ", "Harga baru    "]
    if iterasi == 0:
        print(ListEdit[0] + ": " + ListX[0])
        return 1
    else:
        Tampilkan_Edit(iterasi-1, ListX)
        print(ListEdit[iterasi] + ": " + ListX[iterasi])


def Edit_Diksi():
    Kosong_Counter = 0
    while Kosong_Counter != 1:
        PilihanY = ""
        ListX = []
        PilihanY = input("Nama          : ")
        ListX.append(PilihanY)
        PilihanY = input("Ukuran        : ")
        ListX.append(PilihanY)
        PilihanY = input("Harga         : ")
        ListX.append(PilihanY)
        print ("Mengedit Joran.............")
```

```
    #Cek Kosongan
        Kosong_Counter = CRUD_Anomaly_Detector(ListX)

    #Perbaiki Format
    ListX[1] = (ListX[1] + "cm")
    ListX[2] = ("Rp." + ListX[2])

    Tampilkan_Edit(2, ListX)
    return ListX

def Edit_Joran(DiksiJoran : dict):
    PilihanX = ""
    while PilihanX != "n":
        print("||||FORMAT PENYIMPANAN KATALOG||||")
        print(">>>>Nama, Ukuran, Harga<<<")
        count = 0
        for item in DiksiJoran:
            count += 1
            print(str(count) + ".", DiksiJoran[item][0], DiksiJoran[item][1],
DiksiJoran[item][2])
        PilihanX = input("Pilih nomor item joran yang ingin diedit: ")
        try:
            print ("Data Joran:", DiksiJoran["Joran_" + str(PilihanX)])
            DiksiJoran["Joran_" + str(PilihanX)] = Edit_Diksi()
            print("Nama sukses terganti!")
            print ("apakah anda ingin mengedit lagi?\n" \
                    "ketik 'y' atau huruf selain 'n' jika iya\n" \
                    "ketik 'n' jika tidak")
            PilihanX = input(": ")
        except KeyError:
            print("Jangan masukkan angka negatif, string, atau angka yang
tidak ada didalam dictionary!")
#Pilihan 3
#Pilihan 4
def Remove_Joran():
    PilihanX = ""
    while PilihanX != "n":
        DiksiX = {}
        try:
            PilihanX = input("Pilih nomor item joran yang ingin dihapus dari
list: ")
            KeyX = "Joran_" + str(PilihanX)
            print("Menghapus Joran", _user_data.DiksiJoran[KeyX])
        except KeyError:
            print("Jangan masukkan angka negatif, string, atau angka yang
tidak ada didalam dictionary!")
        else:
```

```
            _user_data.DiksiJoran.pop(KeyX)
            print("Joran telah dihapus!")
            #Menyusun Ulang Urutan Joran
            count = 1
            for x in _user_data.DiksiJoran.values():
                DiksiX["Joran_" + str(count)] = x
                count += 1
            _user_data.DiksiJoran = DiksiX
            print ("apakah anda ingin menghapus lagi?\n" \
                   "ketik 'y' jika iya\n" \
                   "ketik 'n' jika tidak")
            PilihanX = input(": ")
#Pilihan 4
```

# File menu_gui

```
import CRUD
import os
import _user_data
import time
#time dan os = library lain
#Variables

#Menu
def menu_admin():
    #menu_admin
    print ("<||||[Selamat Datang menu admin]||||>")
    Pilihan = ""
    while Pilihan != "6":
        print ("Apa yang ingin anda lakukan?\n" \
            "1. Buat Kategori Joran Baru\n" \
            "2. Lihat Joran Yang Sudah Ada\n" \
            "3. Edit Nama Joran\n" \
            "4. Hapus Joran Dari Toko\n" \
            "5. Bersihkan Layar\n" \
            "6. Keluar Dari Menu Admin")
        Pilihan = input("Pilih Aksi: ")

        if Pilihan == "1": #Menambahkan Joran Baru
            CRUD.Add_Joran(_user_data.DiksiJoran)

        elif Pilihan == "2":
            CRUD.View_Joran(_user_data.DiksiJoran)

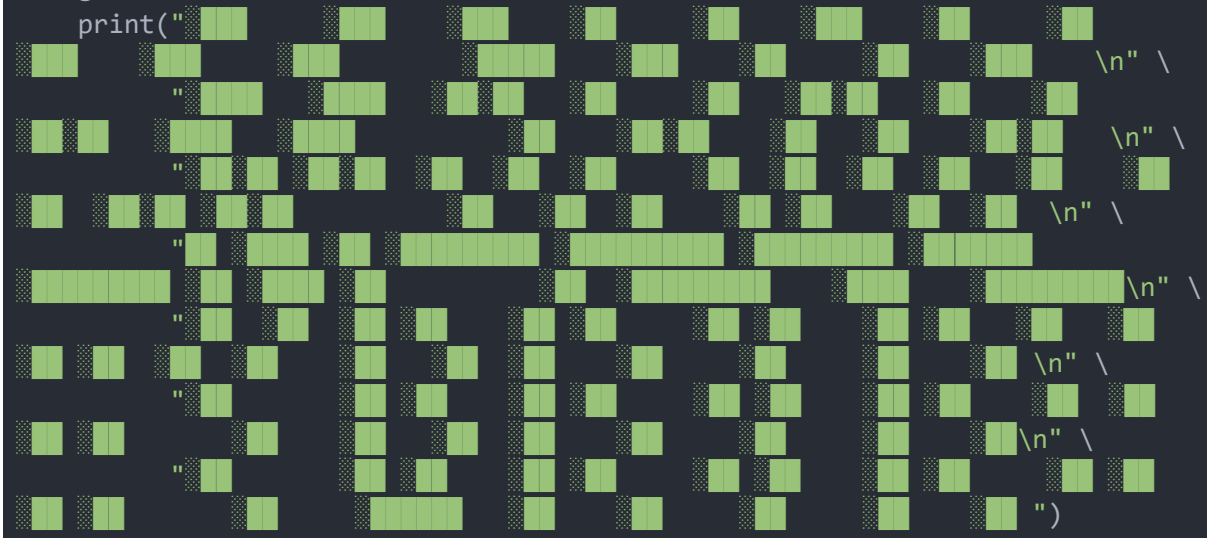
        elif Pilihan == "3": #Edit Joran
            CRUD.Edit_Joran(_user_data.DiksiJoran)
```

```
        elif Pilihan == "4": #Menghapus Joran dari List
            CRUD.Remove_Joran()

        elif Pilihan == "5":
            print("\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n")

        elif Pilihan == "6":
            print("Terimakasih sudah menggunakan program ini !")
        else:
            print("Tolong Masukkan Input Yang Benar!")
        time.sleep(0.25)
        os.system("cls")
    #MENU ADMIN

def menu_user():
    global DiksiJoran
    print("
```



```
    print ("<||||[Selamat Datang di Toko Pancing Mahakam Jaya]||||>")
    Pilihan = ""
    while Pilihan != "3":
        print ("Apa yang ingin anda lakukan?\n" \
               "1. Lihat Katalog Joran Yang Ada\n" \
               "2. Bersihkan Layar\n" \
               "3. Keluar Dari Menu User")
        Pilihan = input("Pilih Aksi: ")
        if Pilihan == "1":
            CRUD.View_Joran(_user_data.DiksiJoran)
        elif Pilihan == "2":
            os.system("cls")
        elif Pilihan != "3":
            print("Silahkan Coba Lagi!")
            time.sleep(0.25)
            os.system("cls")
        time.sleep(0.25)
```

```
        os.system("cls")
    #MENU USER
    os.system("cls")
    print("Terimakasih Sudah Menggunakan Layanan Kami!")

def Dupe_Check(PilihanX : str):
    StatusDuplikat = 0
    for keys in _user_data.DiksiUser.keys():
        if keys != PilihanX:
            StatusDuplikat = 0
        else:
            StatusDuplikat = 1
            print ("Akun dengan nama itu sudah ada, silahkan coba lagi!")
            break
    return StatusDuplikat

def Null_Check(variables : str):
    if variables == "":
        return 1
    return 0

def Create_Account(DiksiUser : dict):
    print("Anda mau buat akun baru? silahkan silahkan!")
    PilihanX = ""
    while PilihanX != "n":
        PilihanX = input("Masukkan Nama     : ")
        PilihanY = input("Masukkan Password : ")
        if Dupe_Check(PilihanX) == 0 and Null_Check(PilihanX) != 1 and
Null_Check(PilihanY) != 1:
            DiksiUser.update({PilihanX : PilihanY})
        elif Null_Check(PilihanX) == 1 or Null_Check (PilihanY) == 1:
            print("Jangan masukkan huruf kosong!")

        PilihanX = input("ketik apapun untuk membuat akun lagi!\n" \
                         "ketik n untuk kembali ke menu: ")
    #Sistem Registrasi

def User_Check():
    PilihanA = ""
    StatusUser = ""
    while PilihanA != "4":
        print("<||||[Selamat Datang Toko Pancing Mahakam Jaya]||||>\n" \
            "Saat ini, anda login sebagai:\n" \
            "1. User\n" \
            "2. Admin \n" \
            "3. Buat Akun\n" \
            "4. Keluar")
        PilihanA = input("Masukkan Pilihan: ")
```

```
        if PilihanA == "1":
            StatusUser = "user"
            break
        elif PilihanA == "2":
            StatusUser = "admin"
            break
        elif PilihanA == "3": #Fitur ini cuma untuk mengisi kuota nilai plus
            Create_Account(_user_data.DiksiUser)
        elif PilihanA == "4":
            StatusUser = "none"
        else:
            print("Jangan Masukkan hal lain. Be a human")
    return StatusUser

def admin_login():
    StatusLogin = 0 #0 false, 1 true
    percobaan = 0
    while percobaan < 3:
        PilihanX = input("Username: ")
        PilihanY = input("Password: ")
        if PilihanX == _user_data.username and PilihanY ==
_user_data.password:
            StatusLogin = 1
            percobaan += 3
            print("
LOGIN BERHASIL")
            print("
\n" \
                "
\n" \
                "
\n" \
                "
\n"
\
                "
\n"
\
                "
\n"
\
                "
")
        else:
            print("Login Gagal, Silahkan coba lagi")
            percobaan += 1
    if percobaan >= 3 and StatusLogin == 0:
```

```
        print("Silahkan renungkan kembali pilihan hidupmu, dan coba lah
mengingat apa passwordmu")
        return
    else:
        menu_admin()
    #ADMIN LOGIN

def user_login():
    count = 0
    _user_data.StatusUser = "noob"
    while count < 3:
        PilihanX = input("Username: ")
        PilihanY = input("Password: ")
        is_duplikat = 0 #photos_printed?
        for keys in _user_data.DiksiUser:
            if keys != PilihanX and _user_data.DiksiUser[keys] != PilihanY:
                _user_data.StatusUser = "noob"
                if is_duplikat != 1:
                    print("Gagal login, silahkan coba lagi!")
                    is_duplikat = 1
                continue
            elif keys == PilihanX and _user_data.DiksiUser[keys] == PilihanY:
                _user_data.StatusUser = "user"
                count += 3
                break
            else:
                _user_data.StatusUser = "noob"
                if is_duplikat != 1:
                    print("Gagal login, silahkan coba lagi!")
                    is_duplikat = 1
        count += 1
    #USER LOGIN

def klasifikasi_user():
    if _user_data.StatusUser == "admin":
        admin_login()
    elif _user_data.StatusUser == "user":
        user_login()
        #Jika login sukses, StatusUser tidak akan berubah
        if _user_data.StatusUser == "user":
            menu_user()
        else:
            print("Gagal Login, Silahkan reboot aplikasi dan buat akun
baru!")
            #Selesai
    elif _user_data.StatusUser == "none":
        print("Terimakasih Sudah Menggunakan Layanan Kami!")
```

**4. Hasil Output**

```
<|||||[Selamat Datang Toko Pancing Mahakam Jaya]|||||>
Saat ini, anda login sebagai:
1. User (Guest)
2. Admin
3. Buat Akun
4. Keluar
Masukkan Pilihan:
```

Gambar 4.1 Tampilan Menu

```
Jangan Masukkan hal lain. Be a human
<|||||[Selamat Datang Toko Pancing Mahakam Jaya]|||||>
Saat ini, anda login sebagai:
1. User (Guest)
2. Admin
3. Buat Akun
4. Keluar
Masukkan Pilihan:
```

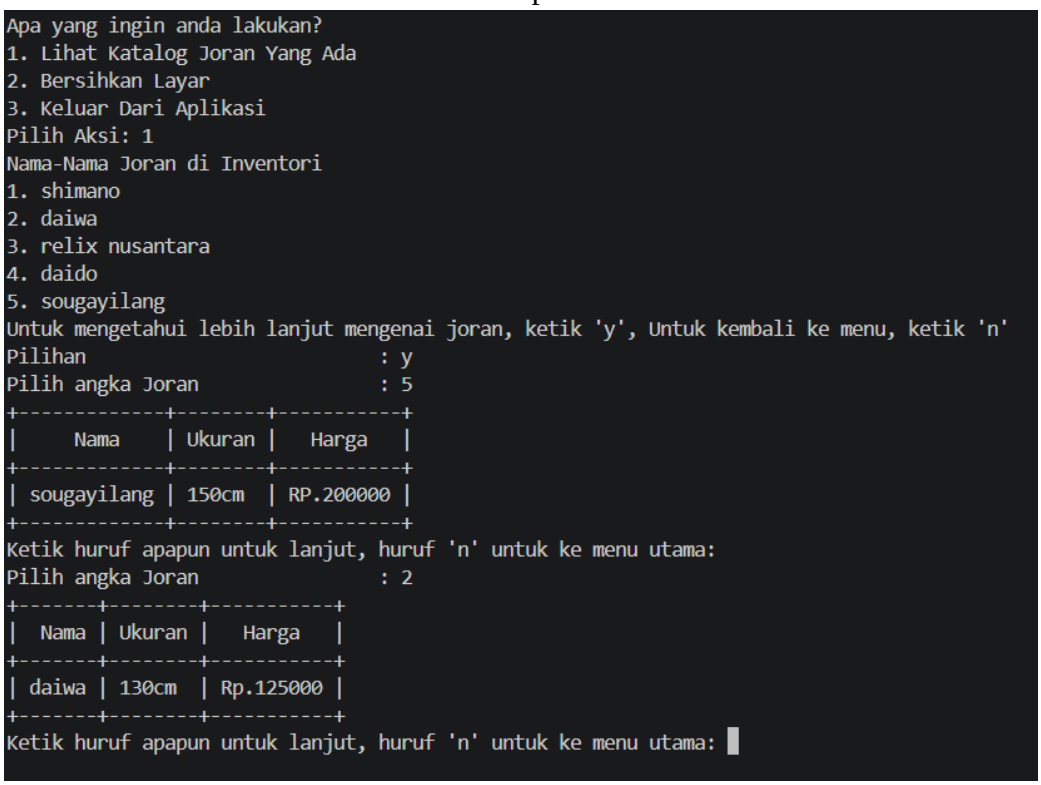
Gambar 4.2 Hasil Output Validasi Kesalahan Input

```
Saat ini, anda login sebagai:
1. User (Guest)
2. Admin
3. Buat Akun
4. Keluar
Masukkan Pilihan: 4
Terimakasih Sudah Menggunakan Layanan Kami!
PS C:\vsCode\Microsoft VS Code>
```

Gambar 4.3 Opsi Keluar




Gambar 4.4 Buat akun dan login user
Termasuk: opsi 3 dan 1




Gambar 4.5 opsi 1 user

```
Apa yang ingin anda lakukan?
1. Lihat Katalog Joran Yang Ada
2. Bersihkan Layar
3. Keluar Dari Aplikasi
Pilih Aksi: 2
```

Gambar 4.6 opsi 2 user (Layar dibersihkan)

```
Apa yang ingin anda lakukan?
1. Lihat Katalog Joran Yang Ada
2. Bersihkan Layar
3. Keluar Dari Aplikasi
Pilih Aksi: 3
Terimakasih Sudah Menggunakan Layanan Kami!
PS C:\vsCode\Microsoft VS Code>
```

Gambar 4.7 opsi 3 user

## Bagian Admin

|
|
|
\\//
\/

```
PS C:\vsCode\Microsoft VS Code> & "C:/Users/Rafa No.2/AppData/Local/Python/bin/python.exe" "c:/Users/Rafa No.2/Documents/Perkuliahan/Praktikum
test_apd_6/2609106015_Rafa Raisyah Al Fattaah_PT-6.py"
<|||||Selamat Datang Toko Pancing Mahakam Jaya|||||>
Saat ini, anda login sebagai:
1. User
2. Admin
3. Buat Akun
4. Keluar
Masukkan Pilihan: 2
Username: John BatuBara
Password: Tongkang123
                                        LOGIN BERHASIL
 ███▄ ▄███▓ ▄▄▄       ██░ ██  ▄▄▄       ██ ▄█▀ ▄▄▄       ███▄ ▄███▓    ▄▄▄██▀▀▀▄▄▄       ▓██   ██▓ ▄▄▄       ▄▄▄
▓██▒▀█▀ ██▒▒████▄    ▓██░ ██▒▒████▄     ██▄█▒ ▒████▄    ▓██▒▀█▀ ██▒      ▒██  ▒████▄      ▒██  ██▒▒████▄    ▒▒▒
▓██    ▓██░▒██  ▀█▄  ▒██▀▀██░▒██  ▀█▄  ▓███▄░ ▒██  ▀█▄  ▓██    ▓██░      ▒██  ▒██  ▀█▄     ▒██ ██░▒██  ▀█▄  ▒▒▒
▒██    ▒██ ░██▄▄▄▄██ ░▓█ ░██ ░██▄▄▄▄██ ▓██ █▄ ░██▄▄▄▄██ ▒██    ▒██    ▓██▄▄▓██▄▄▄▄██    ▒██ ▒ ▒ ▒ ▒ ▒ ▒ ▒ ▒ ▒
▒██▒   ▒▒ ▒ ▒▒▒▒▒   ▒▒ ▒ ▒ ▒ ▒▒▒▒▒   ▒ ▒▒ ▒ ▒ ▒▒▒▒▒   ▒ ▒▒ ▒ ▒ ▒ ▒ ▒ ▒ ▒ 
```

```
<|||||[Selamat Datang menu admin]|||||>
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Aplikasi
Pilih Aksi: 1
Masukkan Nama Joran Baru: joran
Masukkan Panjang Joran (cm):10
Masukkkan Harga Joran (Rp): 1000
Joran Berhasil ditambahkan ke dalam katalog!
apakah anda ingin mengedit lagi?
ketik 'y' atau huruf lain jika iya
ketik 'n' jika tidak: n
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Aplikasi
Pilih Aksi: s
```

Gambar 4.9 Admin opsi 1

```
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Menu Admin
Pilih Aksi: 2
Nama-Nama Joran di Inventori
1. shimano
2. daiwa
3. relix nusantara
4. daido
5. sougayilang
Untuk mengetahui lebih lanjut mengenai joran, ketik 'y', Untuk kembali ke menu, ketik 'n'
Pilihan                         : y
Pilih angka Joran               : 6
Jangan masukkan angka negatif, string, atau angka yang tidak ada didalam dictionary!
Pilih angka Joran               : 5
+-------------+--------+-----------+
|     Nama    | Ukuran |   Harga   |
+-------------+--------+-----------+
| sougayilang | 150cm  | RP.200000 |
+-------------+--------+-----------+
Ketik huruf apapun untuk lanjut, huruf 'n' untuk ke menu utama:
```

Gambar 4.10 Admin opsi 2

```
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Aplikasi
Pilih Aksi: 3
||||FORMAT PENYIMPANAN KATALOG||||
>>>>Nama, Ukuran, Harga<<<
1. shimano 150cm RP.200.000
2. daiwa 130cm Rp.125.000
3. relix nusantara 150cm RP.200.000
4. daido 150cm RP.200.000
5. sougayilang 150cm RP.200.000
Pilih nomor item joran yang ingin diedit:
```

Gambar 4.11 Admin opsi 3

```
Pilih Aksi: 4
Pilih nomor item joran yang ingin dihapus dari list: 1
Menghapus Joran ['JorN', '123', '100000']
Joran telah dihapus!
apakah anda ingin menghapus lagi?
ketik 'y' jika iya
ketik 'n' jika tidak
:
```

Gambar 4.12 Admin opsi 4

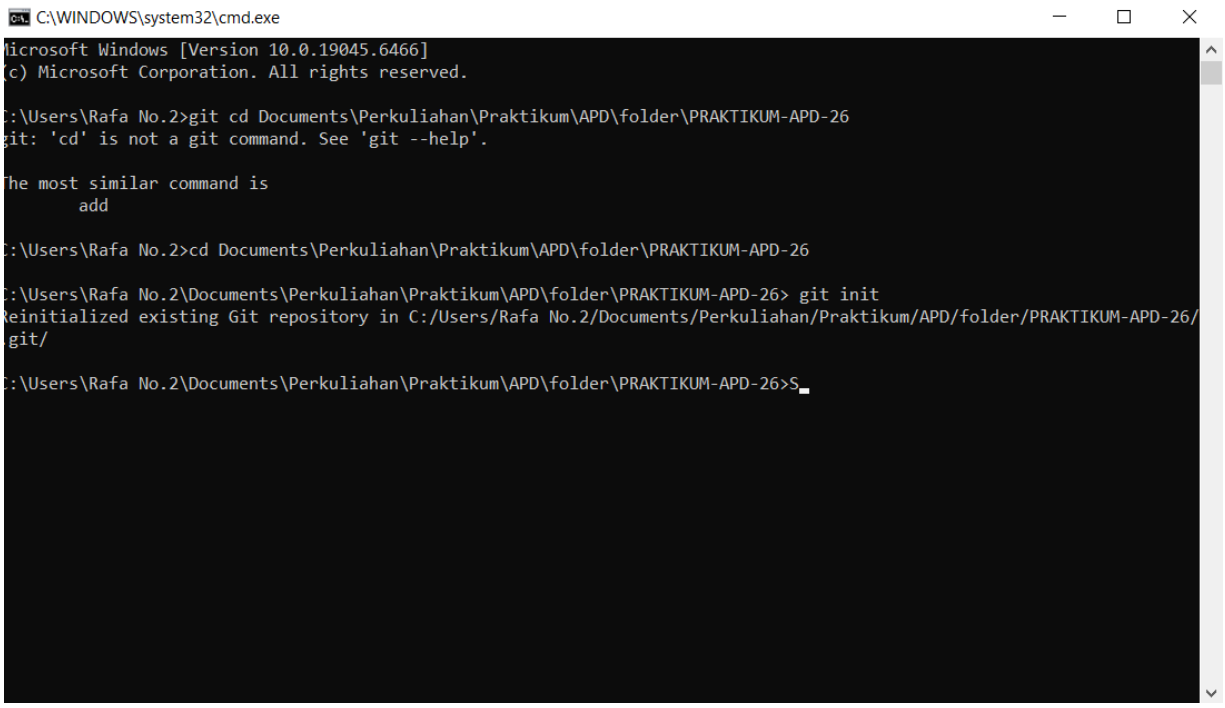
```
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Aplikasi
Pilih Aksi:
```

Gambar 4.13 Admin opsi 5 (Konteks: layarnya udah dibersihkan)

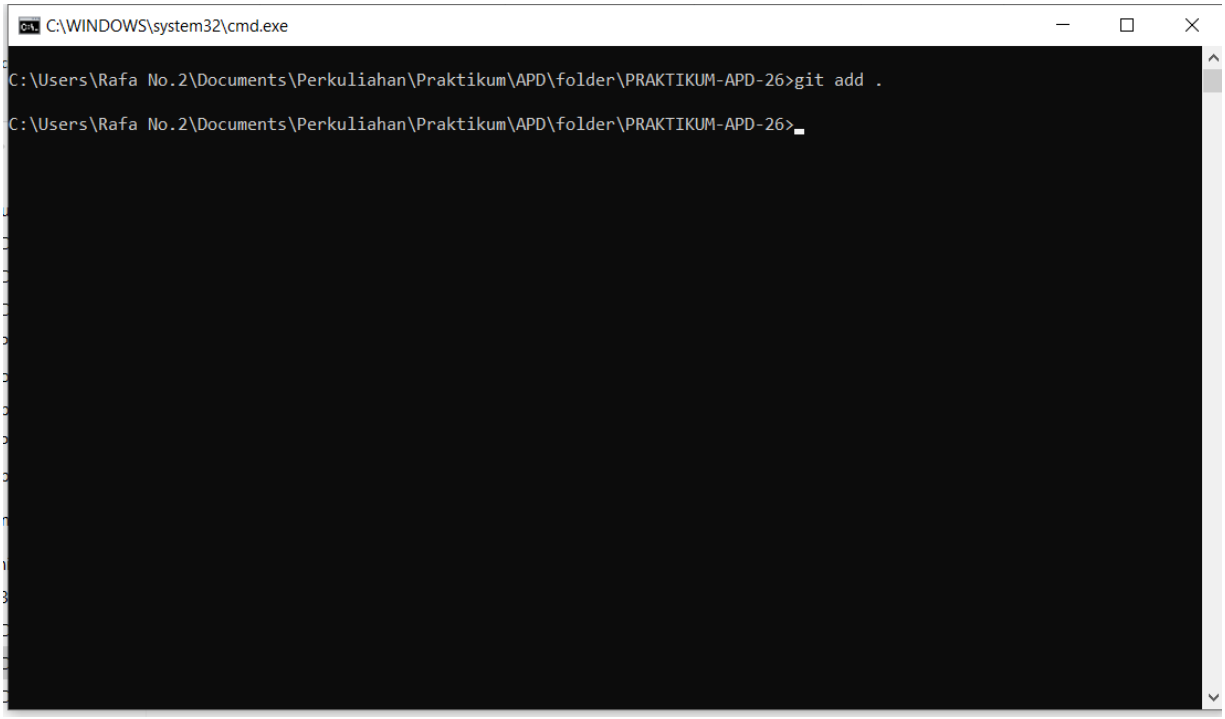
```
Apa yang ingin anda lakukan?
1. Buat Kategori Joran Baru
2. Lihat Joran Yang Sudah Ada
3. Edit Nama Joran
4. Hapus Joran Dari Toko
5. Bersihkan Layar
6. Keluar Dari Aplikasi
Pilih Aksi: 6
Terimakasih sudah menggunakan program ini !
PS C:\vsCode\Microsoft VS Code>
```

Gambar 4.14 Admin opsi 6

## 5. Mengunggah Hasil ke Github

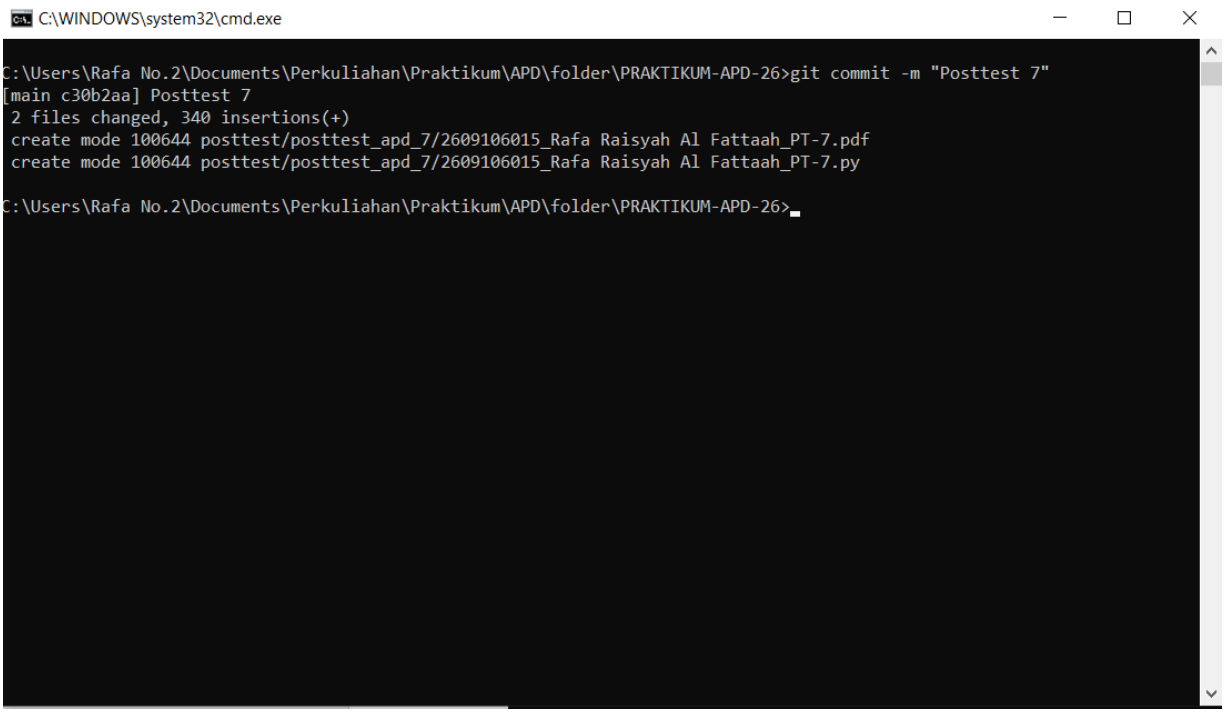



**Git init:**
Mengreinisialisasi repositori lokal di storage



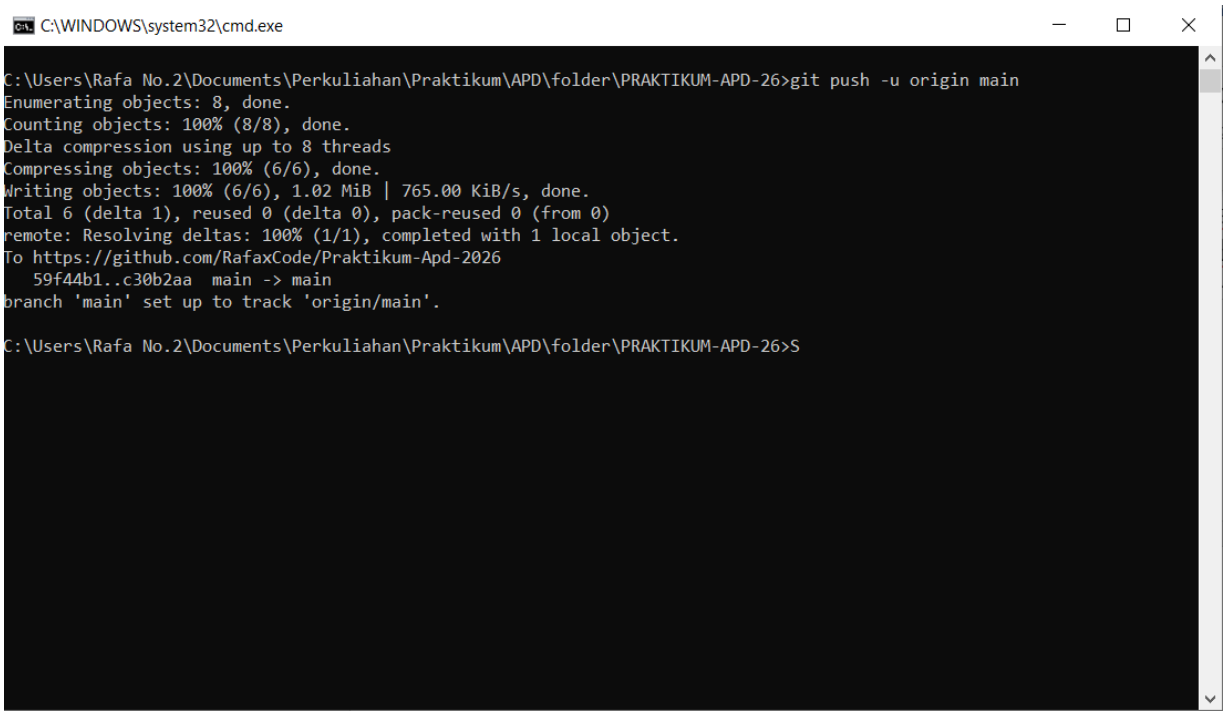

**Git add:**

menambahkan file dan menyiapkannya sebelum dilakukan commit.




**Git commit:**

Melakukkan commit (menyimpan projek pada saat itu). yang disimpan ialah projek file saat itu dan tanggal penyimpanannya. Kegunaan commit ini agar file tersimpan dan projek tetap aman. Fungsi perintah -m ialah sebagai pesan pengingat kepada diri saya di masa depan mengenai commit proyek saat ini.




**Git push:**

Mendorong commit/hasil simpanan save yang sudah di commit atau di save ke dalam repo github, spesifiknya, repo Praktikum-Apd-2026 di file posttest, di dalam filenya lagi posttest8.

Praktikum-Apd-2026 -> posttest -> posttest8